

可持久化数据结构

我们前面做过的数据结构题一般都是在上一次修改之后继续修改或者查询，我们只需要维护维护每一次修改完之后的数据结构的形态就可以了，也就是说我们只需要维护一棵树。

但是有些题目的问题并不是这样，他可能会在某个历史版本上做修改或者进行询问。比如：在前面第 k 个操作的基础上修改 x 的值，询问第 k 个操作后 x 这个位置的值。

那么对于这一类问题我们又该如何处理呢？

可持久化数据结构

题目描述

如题，你需要维护这样的一个长度为 N 的数组，支持如下几种操作

1. 在某个历史版本上修改某一个位置上的值
2. 访问某个历史版本上的某一位置的值

此外，每进行一次操作（对于操作2，即为生成一个完全一样的版本，不作任何改动），就会生成一个新的版本。版本编号即为当前操作的编号（从1开始编号，版本0表示初始状态数组）

输入格式

输入的第一行包含两个正整数 N, M ，分别表示数组的长度和操作的个数。

第二行包含 N 个整数，依次为初始状态下数组各位的值（依次为 a_i ， $1 \leq i \leq N$ ）。

接下来 M 行每行包含3或4个整数，代表两种操作之一（ i 为基于的历史版本号）：

1. 对于操作1，格式为 $v_i \ 1 \ loc_i \ value_i$ ，即为在版本 v_i 的基础上，将 a_{loc_i} 修改为 $value_i$
2. 对于操作2，格式为 $v_i \ 2 \ loc_i$ ，即访问版本 v_i 中的 a_{loc_i} 的值，生成一样版本的对象应为 v_i

可持久化数据结构

因为我们需要在每个历史版本上进行操作，那么很容易想到一种很暴力的做法，也就是保留每一次修改之后的树的形态，也就是维护一片森林。那么第 k 棵线段树就表示第 k 次操作之后的树的形态，那么下一次操作如果是在第 k 次操作的基础上继续操作，那么我们直接从第 k 棵树的根节点开始访问。

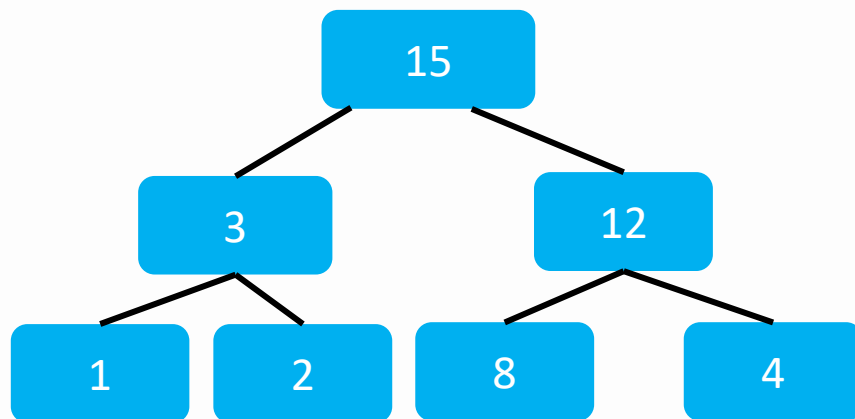
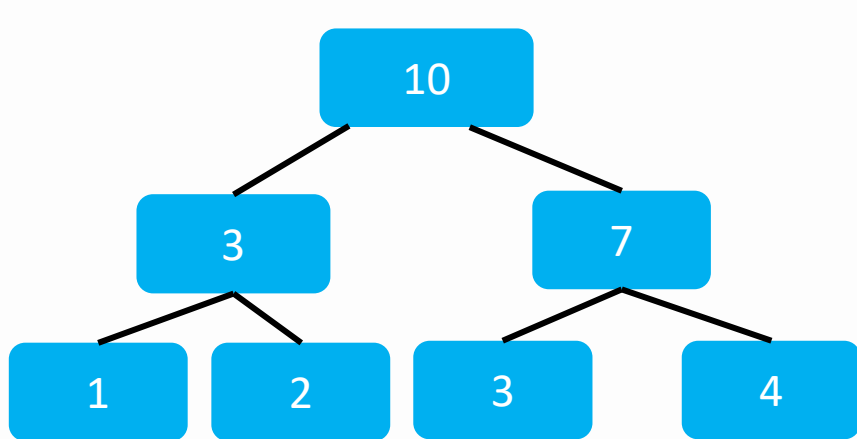
但是这样做最大的问题就是时间和空间，每一次操作都要建一棵完整的数，那么每一次操作时间复杂都是 $O(n)$ ，所以时间和空间是不满足要求的。

我们仔细观察一下相邻的两棵树

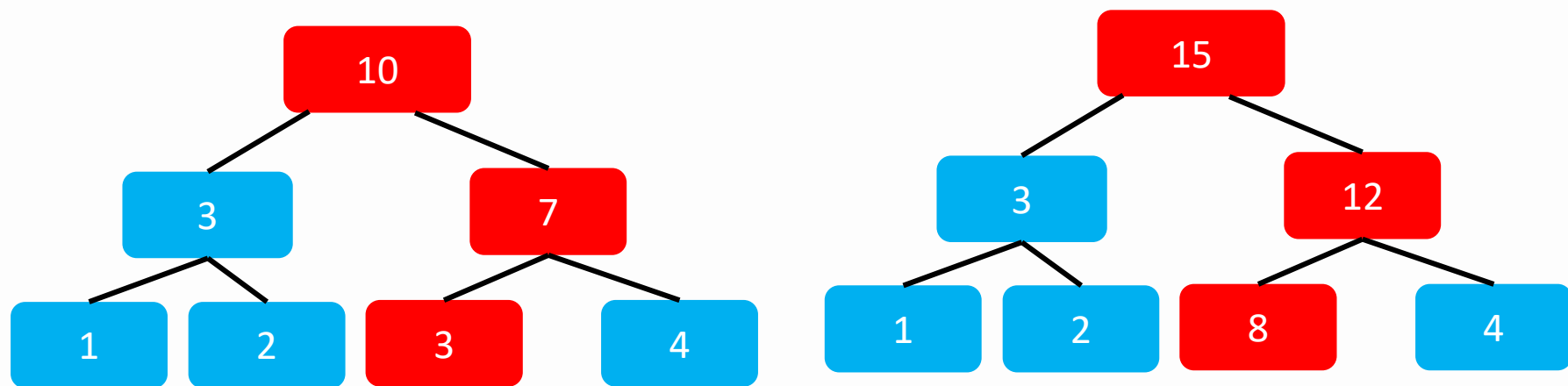
可持久化数据结构

假设我们需要用线段树来维护一次操作，那么我们来看下相邻两棵线段树的区别和联系。

原始数组 1 2 3 4 我们需要维护区间和
下一次操作：修改第3个数为8

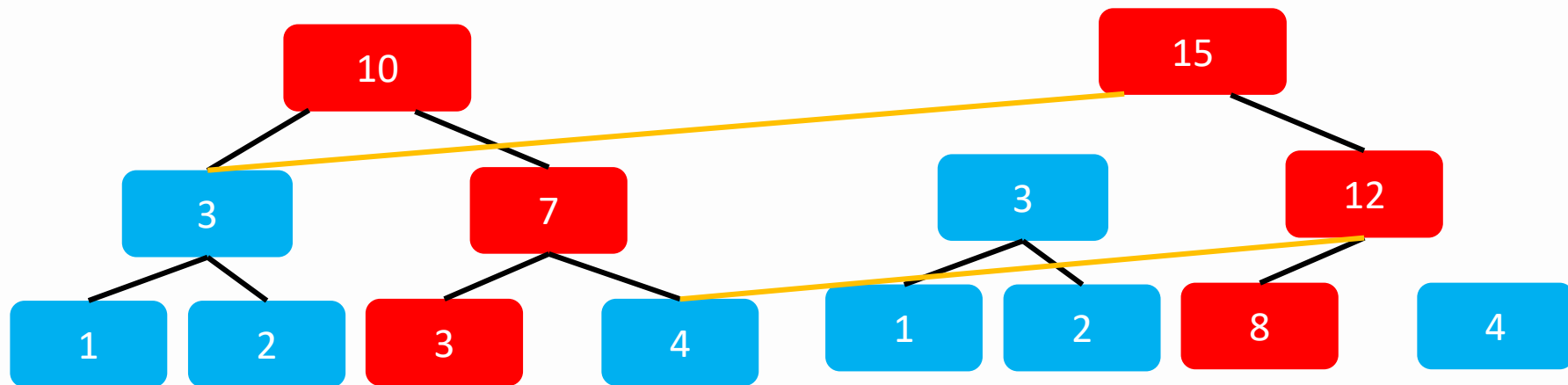


可持久化数据结构



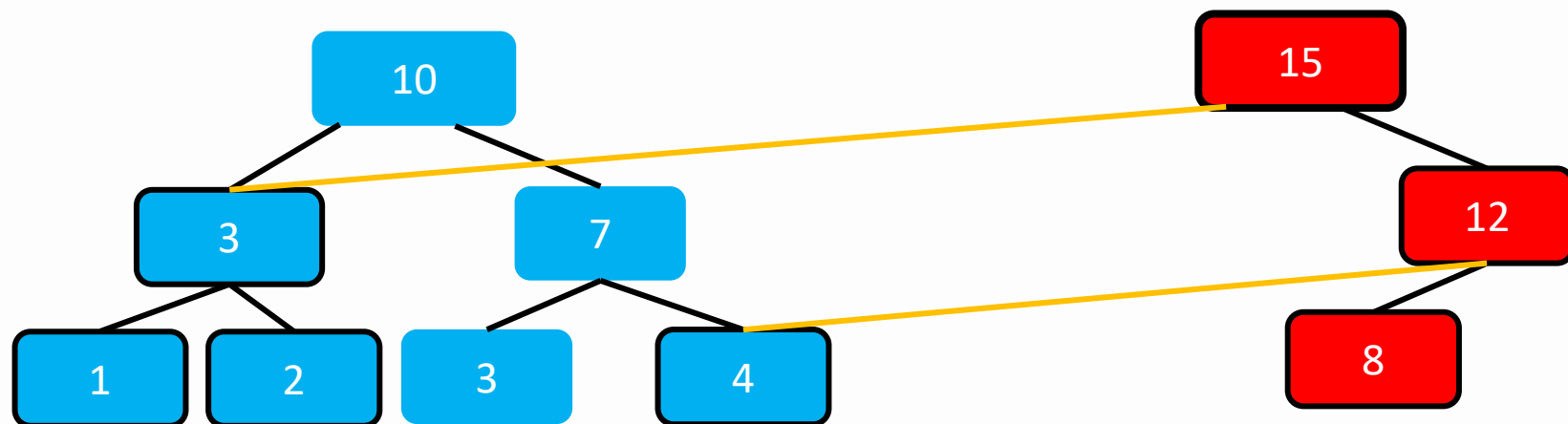
我们很容易发现，对于单点修改，每次修改前面的两颗线段树并不是完全不一样，而是有相同的部分，也有不同的部分，每次修改只是修改了一条链，即根节点到该节点所在叶子这一条链，对于其他部分，我们都是不需要修改的。所以我们并不需要重新建一棵完整的树。

可持久化数据结构



比如在该题中，我们修改第3个数的值，从根节点开始遍历，3号节点在根节点的右子树，那么根节点的左子树不需要修改，所以我们可以直接让新数的根节点的左儿子指向上一棵树的左儿子，左子树后面都不用去管了。右儿子需要修改，新开一个节点，到了右儿子后，我们发现3在他的左二子，那么右儿子直接继承，左二子新建节点。

可持久化数据结构



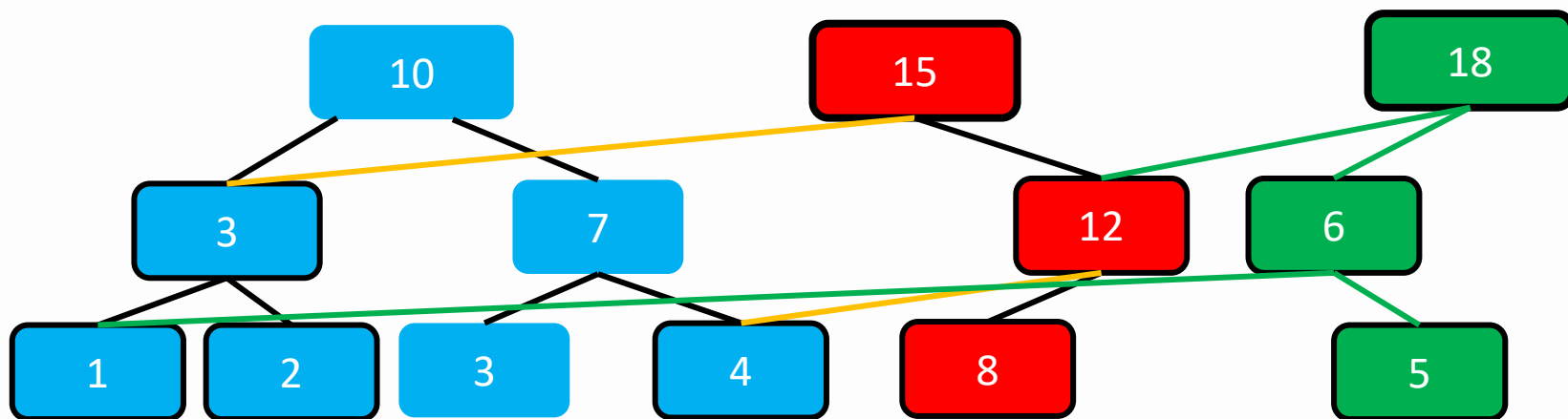
这样我们每次修改一个节点的时候，新建的节点数只有一条链的长度，也就是最多修改 $\log n$ 个节点，也只会访问 $\log n$ 个节点。图中带黑色框的部分就是新的线段树。仍然是一棵完整的线段树，但是其中的部分未修改的节点仍然是使用了前面已有的节点。

可持久化数据结构

原始数组 1 2 3 4

第一次修改 第3个数为8

第二次修改 第2个数为5



可持久化数据结构

可持久化数据结构原理特别简单，代码也不长，只是在原有的数据结构上做一个简单的修改。

空间问题：因为可持久化数据结构每次会新增一条链，即 $\lceil \log n \rceil$ 个点，那么如果对于一个长度为 n 的数组，有 n 次操作，那么 n 次操作完之后的点的个数为 $4*n + n*\log n$ ，对于10w个点，一般数组开20+就可以了，100w开30，一般来说为了不出问题，一般都开30倍。、

可持久化数据结构只能采用动态开点的形式，如果采用 $id*2$ 的形式，肯定爆空间。

可持久化数据结构

```
int build(int l,int r)//动态建树
{
    int now=++k;
    if(l==r)
    {
        tree[now].val=a[l];
        return now;
    }
    int mid=(l+r)/2;
    tree[now].l=build(l,mid);
    tree[now].r=build(mid+1,r);
    return now;//把值返回给父亲节点
}
root[0]=build(1,n);
```

可持久化数据结构

```
for(int i=1;i<=m;i++)
{
    scanf("%d%d",&old,&opt);//old表示历史版本
    if(opt==1)//修改
    {
        scanf("%d%d",&x,&y);
        root[i]=update(root[old],1,n,x,y);
    }
    else if(opt==2)//查询
    {
        scanf("%d",&x);
        printf("%d\n",query(root[old],1,n,x));
        root[i]=root[old];//查询要继承历史版本
    }
}
```

可持久化数据结构

```
int update(int pre,int l,int r,int x,int y)
{//两颗线段树同时跳转，始终保持在相同区间，这样才能继承
    int now=++k;
    tree[now].l=tree[pre].l;
    tree[now].r=tree[pre].r;//区间相同
    if(l==r)
    {
        tree[now].val=y;
        return now;//边更新边建树，所以要返回值
    }
    int mid=(l+r)/2;
    if(x<=mid)
        tree[now].l=update(tree[pre].l,l,mid,x,y);
    else
        tree[now].r=update(tree[pre].r,mid+1,r,x,y);
    return now;
}
```

可持久化数据结构

```
int query(int pre,int l,int r,int x)
{ // 在历史版本上查询, 和普通线段树的查询一样
    if(l==r)
        return tree[pre].val;
    int mid=(l+r)/2;
    if(x<=mid)
        return query(tree[pre].l,l,mid,x);
    else
        return query(tree[pre].r,mid+1,r,x);
}
```



可持久化数据结构

像这种询问历史版本的数据结构的题特别少，但是可持久化数据结构除了可以做这种很明显的带有历史版本的数据结构的题之外，还有一个很常见的应用——询问静态区间第 k 大。



区间第K大

给定N个整数构成的序列，将对于指定的闭区间查询其区间内的第K小值。

输入：

5 5

25957 6405 15770 26287 26465

2 2 1

3 4 1

4 5 1

1 2 2

4 4 1

输出：

6405

15770

26287

25957

26287

区间第K大

我们以前做的题都是用线段树求解区间极大(小)值，最多也就是次大(小)值，但是对于第K大的值，我们还是第一次遇到。因为K是不确定的，所以线段树肯定维护不了，那我们考虑一下平衡树是否可以实现。

平衡树建树要么依据区间编号建树，要么依据权值大小建树。如果我们保证区间有序，那么查询的区间是很容易分离出来，但是区间内的答案却不是按照权值排序的；如果我们保证权值有序，那么我们就没有办法把需要查询的区间分离出来，自然也就没有办法求解区间第K大。

所以我们需要学习一种新的数据结构来解决问题。

前缀第K大

我们先思考一个简单的题：依次给定 n 个数，再给定 m 个询问，问前 i 个数中第 k 大的数是多少，保证每次询问的位置递增。

输入：

1 2 3 4 5

3

2 2

4 2

5 3

输出：

1

3

3

前缀第K大

当然，这道理可以用我们学过的平衡树很容易解决，但是实际上连平衡树都不需要，一棵线段树就可以解决了。

我们可以用权值线段树来解决、先建一颗 n 个结点的空树，对于查询前 i 个数第 k 大，我们可以先把前 i 个数的权值当做编号放进去，每个结点记录当前子树有多少个数，在查询的时候，参考平衡树的查询第 k 大的方式。

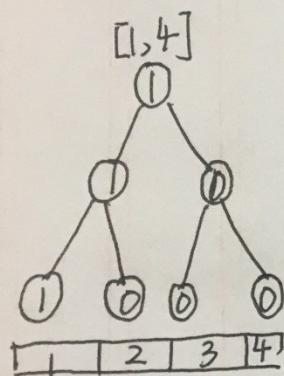
主席树

所以我们知道如果是前缀第 k 大，我们可以用权值线段树来查询，但是如果是区间呢？如果我们需要知道每次查询的区间内到底有哪些数，那我们就需要可持久化数据结构来维护，简单来说，也就是如果我们还是用线段树来维护，那么我们需要知道每次插入一个数之后的线段树的形态。这样我们就可以用两个不同时间段的线段树相减就得到了我们需要的区间。因为相减之后还存在的数就表示在该区间内出现的数。

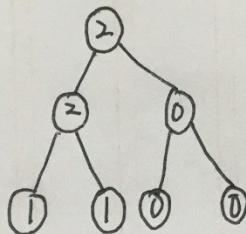
主席树

1 2 3 4

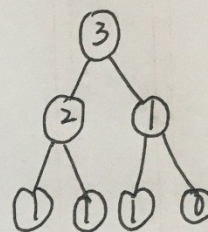
1. 插入 1.



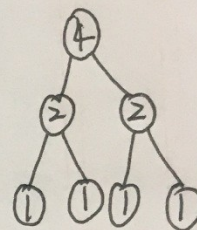
2. 插入 2



3. 插入 3.

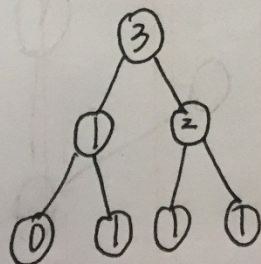


4. 插入 4.



查询区间 $[2, 4]$ 先找到第一棵树 再找到第 4 棵树 相减.

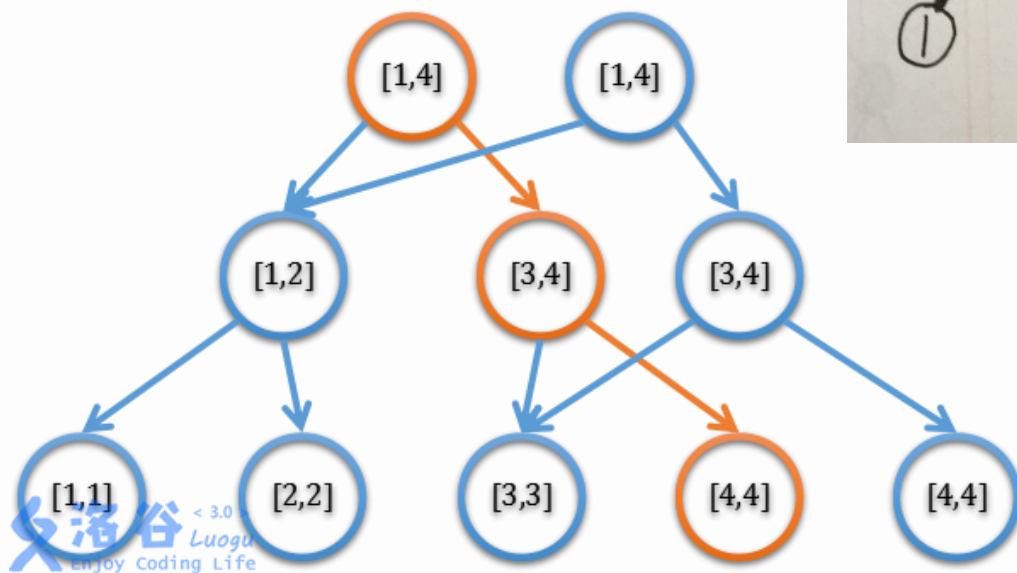
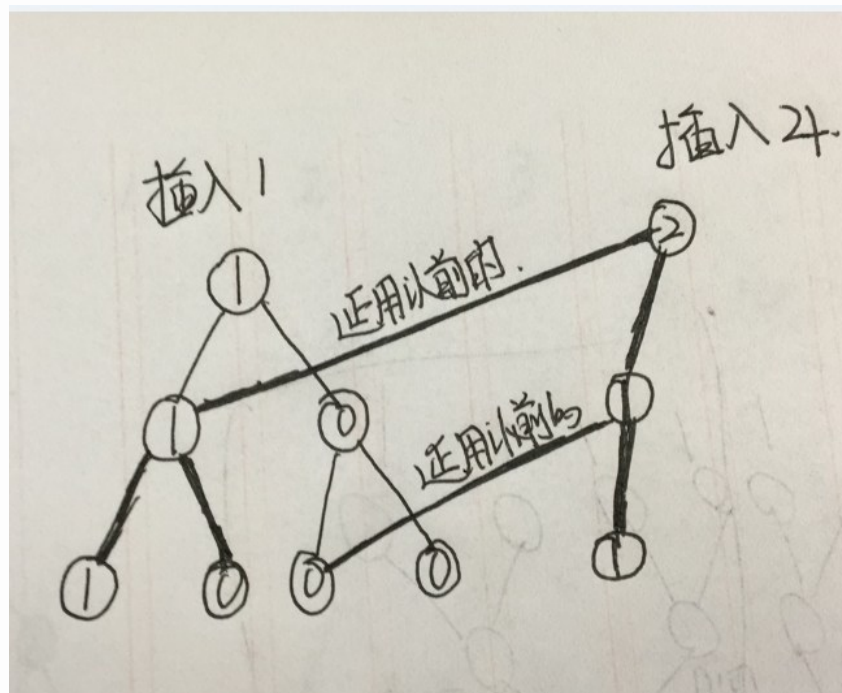
相减之后树的结构即为区间 $[2, 4]$ 出现的数.



主席树

这样做是可以解决问题，但是这样做的时间复杂度和空间复杂度都是我们不能接受的，因为对于 n 个数来说，需要开 n 棵线段树，时间复杂度和空间复杂度都是 $O(n^2)$ 的，所以我们需要优化。我们仔细观察一下相邻的两棵树，我们会发现，相邻的两棵树只有一条链不相同，即修改的那个节点向上的那条链，其他地方都是一样的。所以我们没有必要再开一颗完整的线段树，只需要在上一棵树的基础上重新增加一条链就可以了，剩下的不变的地方就沿用以前的就可以了。这样空间复杂度只需要 $N+N\log N$ ，对于100000的数据， $*1 \ll 5$ 就够了，时间复杂度也是 $N\log N$ 。

主席树



实现过程

第一步：因为我们要建权值线段树，但是数的范围比较大，所以我们需要先离散化，以免空间不够。注意需要因为要知道原来的值，所以开双向映射。

第二步：初始化建树，先建一棵空的线段树，为了节省空间已经后期好更改左右儿子的指针，所以主席树建树采用线段树动态开点来实现。因为主席树实际上一片森林而不是一棵树，所以我们需要知道每一棵树的根结点，所以开一个root数组来记录每次插入数之后的根结点新生成的树的根结点的编号。

实现过程

```
int build(int l,int r)
{
    int root=++k; //动态开点
    tree[root].sum=0; //目前是空树
    if(l<r)
    {
        int mid=(l+r)/2;
        tree[root].l=build(l,mid);
        //左右子树开好点之后再把该点编号返回给指向他的结点
        tree[root].r=build(mid+1,r);
    }
    return root;
}
```


实现过程

第三步：把第 i 个数当做第 i 次插入的数，分别在上一棵树的基础上建立 n 个线段树，注意空间需要开 $n \log n$

```
int updata(int pre,int l,int r,int x)//pre表示上一棵树的根结点
{
    int root=++k;
    tree[root].l=tree[pre].l;//每课线段树的区间范围是一样的
    tree[root].r=tree[pre].r;
    tree[root].sum=tree[pre].sum+1;//比上一次多了一个数
    if(l<r)
    {
        int mid=(l+r)/2;
        if(x<=mid)//如果新增节点在左边，则右边子树不变
            tree[root].l=updata(tree[pre].l,l,mid,x);
        else
            tree[root].r=updata(tree[pre].r,mid+1,r,x);
    }
    return root;
}
```

实现过程

第四步：查询区间 $[l, r]$ 第 K 大的数，即用第 r 棵线段树的每个根结点-第 $(l-1)$ 棵线段树的每个根结点，这样得到的树的形态就是区间 $[l, r]$ 之间的数组成的线段树。再按照权值线段树的方法分别根据左右区间的数的数量来找到第 K 大的值是多少。

```
int query(int u,int v,int l,int r,int ki)
{ //u表示(l-1),v表示r
  if(l>=r)//表示找到了第ki大的数
    return l;
  int x=tree[tree[v].l].sum-tree[tree[u].l].sum;
  int mid=(l+r)/2;
  if(x>=ki)//表示在右边区间
    return query(tree[u].l,tree[v].l,l,mid,ki);
  else
    return query(tree[u].r,tree[v].r,mid+1,r,ki-x);
}
```

可持久化并查集

我们以前学习的并查集只可以合并而不可以撤销，如果需要维护一个既可以合并又可以撤销的的并查集可以用可持久化并查集来维护。

在并查集的合并操作中有一个比较重要的操作叫路径压缩，实际上我们的并查集可以看做一棵树，但是如果使用了路径压缩，这棵树的形态就会发生改变，不仅仅只是左右子树中之一会变化。所以可持久化并查集不能使用路径压缩。

那么为了保证并查集的效率，我们应该如何合并呢？其实并查集的效率 and 并查集的深度有关，只要保证合并之后并查集的深度最小就可以了，所以我们可以用启发式合并(按秩合并)，即把深度小的并查集的根结点，连接到深度大的并查集的根结点去。

可持久化并查集

```
int f1=getf(x);
int f2=getf(y); //求祖先
if(deep[f1]>deep[f2])
{
    int tmp=f1;f1=f2;f2=tmp;
}
fa[f1]=f2; //把f1合并到f2
if(deep[f1]==deep[f2])
    deep[f2]++; //相等, 深度+1, 否则深度不变
```

可持久化并查集

可持久化并查集可以用可持久化线段树来实现，对于线段树的每一个位置，我们存放该结点所在集合的根结点的信息。

每次并查集合并的时候，我们只需要把其中一个集合的根结点的父亲结点更新为另一个集合的父亲结点就可以了。和前面线段树的操作一样，对从根结点到这个叶子结点这一条链新开一串结点来更新其父亲结点。更新完父亲结点之后，如果这两个集合本来深度就相同，那么还需要更新一下 f_2 所在集合的深度。

在并查集中求祖先实际上就是线段树的查询操作，在线段树上找到其祖先，如果是集合的根，就不再找了，如果不是，那么继续在线段树上找，直到满足条件为止。

可持久化并查集

```
int build(int l,int r)
{
    int now=++k;
    if(l==r)
    {
        tree[now].fa=l; // 最开始父亲结点都是它本身
        return now;
    }
    int mid=(l+r)/2;
    tree[now].l=build(l,mid);
    tree[now].r=build(mid+1,r);
    return now;
}
```

可持久化并查集

```
int update(int pre,int l,int r,int x,int y)
{
    int now=++k;
    tree[now]=tree[pre];
    if(l==r)
    {
        tree[now].fa=y; //更新该点的父亲结点
        deep[now]=deep[pre]; //先继承根结点深度
        return now;
    }
    int mid=(l+r)/2;
    if(x<=mid)
        tree[now].l=update(tree[pre].l,l,mid,x,y);
    else
        tree[now].r=update(tree[pre].r,mid+1,r,x,y);
    return now;
}
```

```
root[i]=update(root[i-1],1,n,tree[f1].fa,tree[f2].fa);
```

可持久化并查集

```
int getf(int now,int x)
{
    int fx=query(now,1,n,x);
    if(x==tree[fx].fa)
        return fx;
    else
        return getf(now,tree[fx].fa);
}

int f1=getf(root[i-1],x);
int f2=getf(root[i-1],y);
if(tree[f1].fa==tree[f2].fa)
{
    root[i]=root[i-1];
    continue;
}
if(deep[f1]>deep[f2])
{
    int tmp=f1;f1=f2;f2=tmp;
}
root[i]=update(root[i-1],1,n,tree[f1].fa,tree[f2].fa);
if(deep[f1]==deep[f2])
    add(root[i],1,n,tree[f2].fa);
```


可持久化并查集

```
void add(int now,int l,int r,int x)
{
    //当两个并查集深度一样的时候, f2所在并查集深度+1
    if(l==r)//在前面更新父亲结点的时候结点已经添加了
    {
        deep[now]++;
        return ;
    }
    int mid=(l+r)/2;
    if(x<=mid)
        add(tree[now].l,l,mid,x);
    else
        add(tree[now].r,mid+1,r,x);
}
```

可持久化数据结构

可持久化数据结构的维护不仅仅可以用多颗线段树来维护，也可以用多棵平衡树来实现，线段树可以可持久化的原因是因为每次修改只会修改一条链，对于其他部分均沿用以前的节点，所以同样的，如果一个数据结构要支持可持久化，那么他必须要满足每次修改只修改一条链这个条件。否则这种数据结构就不支持可持久化。

平衡树的实现方法有很多种，我们学习过的有带旋转的Splay，和不带旋转的FHQ，SPLAY每次旋转树的形态会发生很大的变化，而FHQ每次合并只会去其中一个子树中去合并，所以SPLAY不支持可持久化，而FHQ支持可持久化，所以设计到可持久化平衡树的题都用FHQ来实现。

可持久化数据结构

题目描述

您需要写一种数据结构（可参考题目标题），来维护一个可重整数集合，其中需要提供以下操作（**对于各个以往的历史版本**）：

- 1、插入 x
- 2、删除 x （若有多个相同的数，应只删除一个，**如果没有请忽略该操作**）
- 3、查询 x 的排名（排名定义为比当前数小的数的个数 $+1$ ）
- 4、查询排名为 x 的数
- 5、求 x 的前驱（前驱定义为小于 x ，且最大的数，**如不存在输出 $-2^{31} + 1$** ）
- 6、求 x 的后继（后继定义为大于 x ，且最小的数，**如不存在输出 $2^{31} - 1$** ）

和原本平衡树不同的一点是，每一次的任何操作都是基于某一个历史版本，同时生成一个新的版本。（操作3, 4, 5, 6即保持原版本无变化）

每个版本的编号即为操作的序号（版本0即为初始状态，空树）

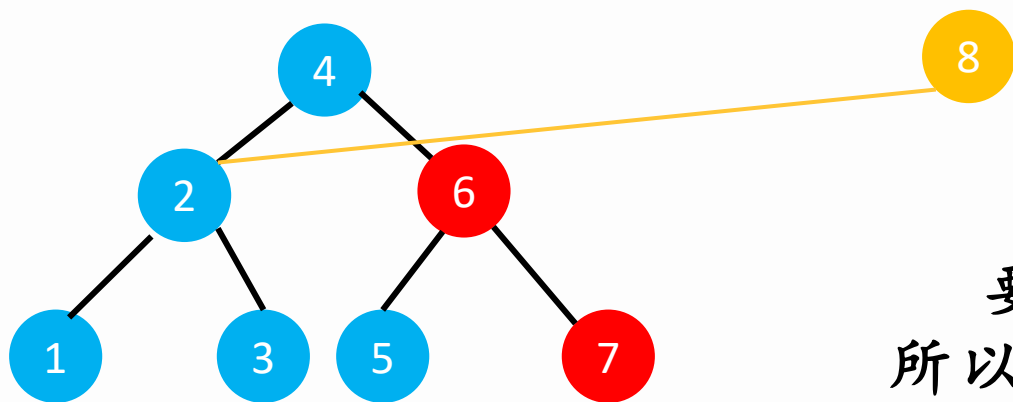


可持久化数据结构

FHQ Treap的核心操作只有两个，`split`和`merge`，对于`split`操作，每次要么去左子树分裂，要么去右子树分裂，那么分裂刚好符合可持久化的特点，每次只修改左右儿子中的其中一个，那么可以实现。对于`merge`操作，每次合并要么去左子树合并，要么去右子树合并，也符合条件，所以FHQ很容易就实现可持久化。



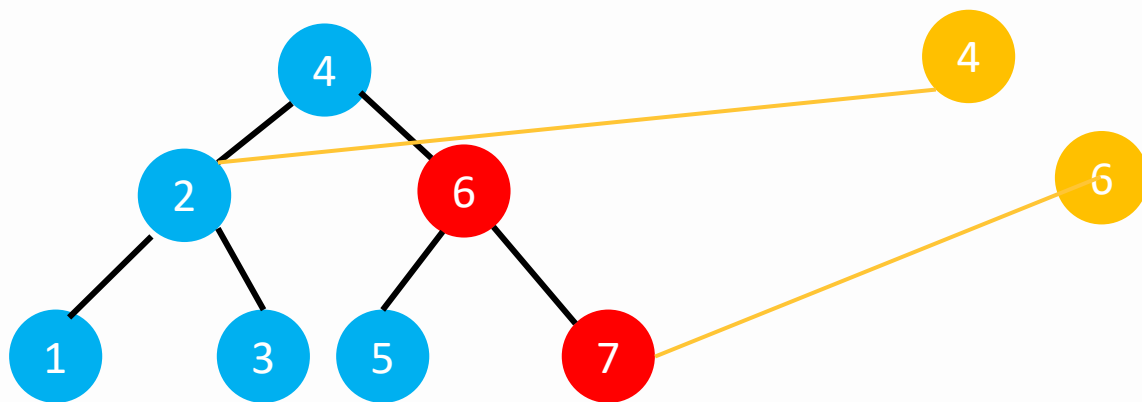
可持久化数据结构



我们按照 $x=5$ 来分裂

要分裂数 x 在4的右子树，所以左子树为根，继续去右子树分裂，所以我们新开一个节点，让他的左儿子等于原根节点的左儿子。

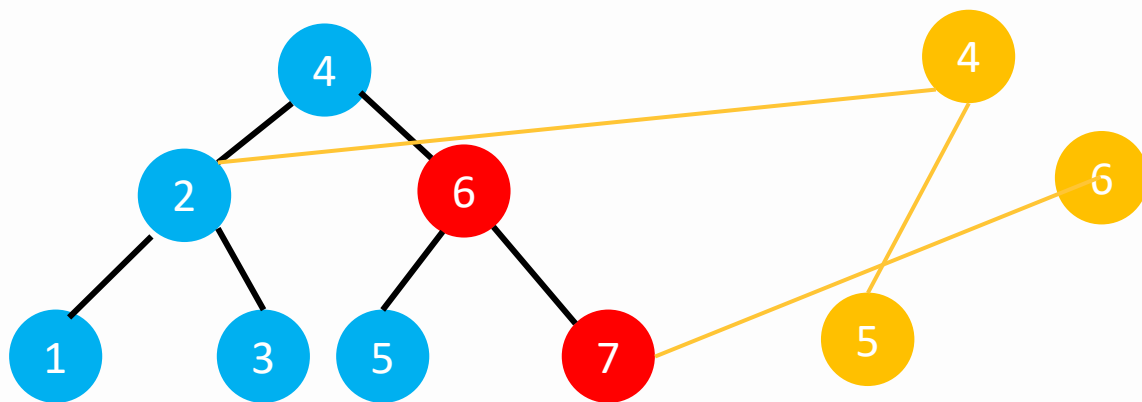
可持久化数据结构



我们按照 $x=5$ 来分裂

要分裂数 x 在6的左子树，那么6就是右子树的根，继续去左子树分裂，为了保证原树不变化，所以我们新开一个权值为6的节点，并且继承原6号节点的右儿子。

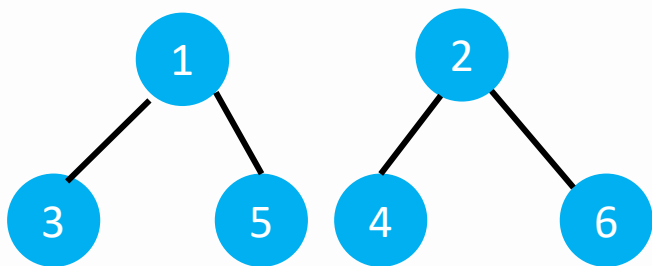
可持久化数据结构



我们按照 $x=5$ 来分裂

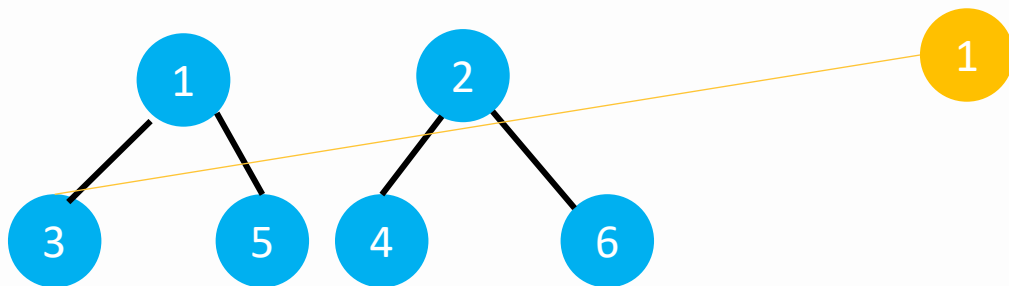
要分裂数 x 在5的右子树，那么5就是左子树的根，继续去右子树分裂，并且成为上一个父亲节点(4)的右儿子。因为平衡树上要遍历到的节点的子树肯定会变化，所以都要新开节点。

可持久化数据结构



同样，合并操作也是一样，因为分裂原树不是真的分裂，所以合并的时候也不能把原树直接合并，好像对涉及子树需要修改的点新增节点，并且继承其中的一个儿子，新一个儿子递归新建。

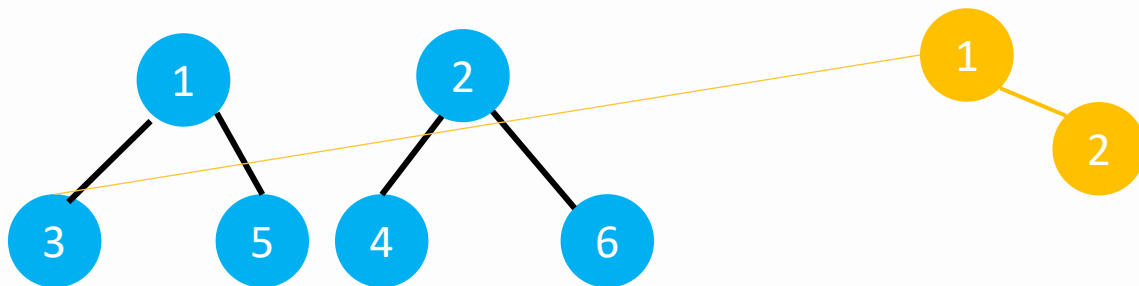
可持久化数据结构



中间的数字表key
值，真实值肯定是
左边树都小于右边

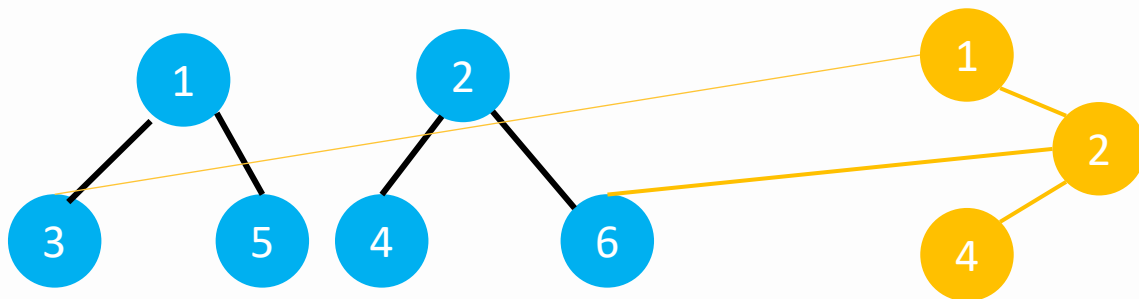
节点1的key比2号节点小，所以1号节点为根，新增一个节点1，节点2肯定在节点1的右子树，所以继承左子树。

可持久化数据结构



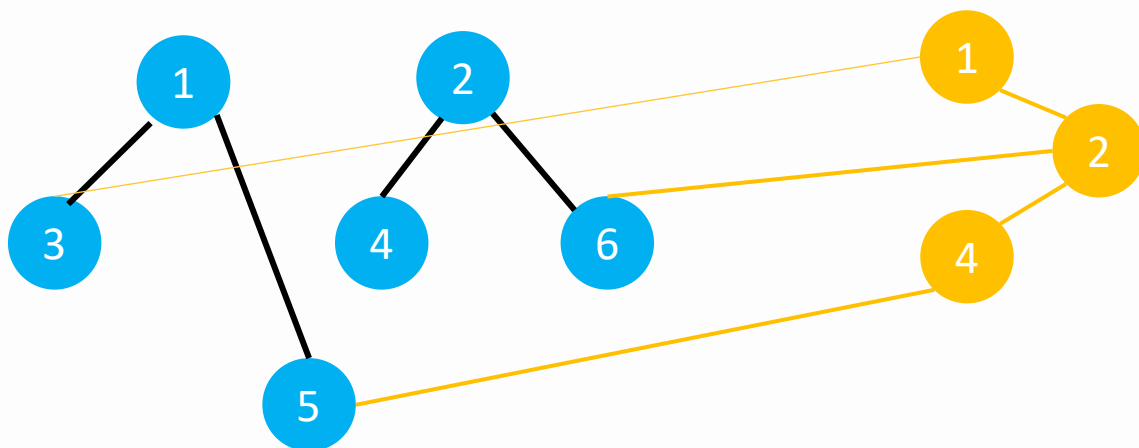
节点5的key比节点2的key大，所以节点2为根，节点5是节点2的左子树，所以新增一个节点2，并且继承右子树。

可持久化数据结构



节点5的key比节点4的key大，所以节点4为根，节点5是节点4的左子树，所以新增一个节点4，并且继承右子树。

可持久化数据结构



右子树已经不存在节点了，直接把左子树的节点连接上去就可以了。

可持久化数据结构

```
root[i]=root[old];//先继承, 也可以不这样写
if(opt==1)//插入x
{
    split(root[i],x,l,r);
    p=add(x);
    root[i]=merge(merge(l,p),r);
}
else if(opt==2)
{
    split(root[i],x,l,r);
    split(l,x-1,l,p);
    p=merge(tree[p].l,tree[p].r);
    root[i]=merge(merge(l,p),r);
}
```

可持久化数据结构

```
void split(int id,int x,int &l,int &r)
{
    if(!id)
    {
        l=r=0;
        return ;
    }
    if(x<tree[id].val)//x在左子树
    {
        r=++k;//右子树的根确定了, 新开一个节点
        tree[r]=tree[id];//继承
        split(tree[r].l,x,l,tree[r].l);
        update(r);
    }
    else//x在右子树
    {
        l=++k;
        tree[l]=tree[id];
        split(tree[l].r,x,tree[l].r,r);
        update(l);
    }
}
```

可持久化数据结构

```
int merge(int x,int y)
{
    if(!x||!y)
        return x+y;
    if(tree[x].key<=tree[y].key)//x为根
    {
        int now=++k;
        tree[now]=tree[x];//新开一个x节点并且继承
        tree[now].r=merge(tree[now].r,y);
        update(now);
        return now;
    }
    else
    {
        int now=++k;
        tree[now]=tree[y];
        tree[now].l=merge(x,tree[now].l);
        update(now);
        return now;
    }
}
```



练习题

- 1: 洛谷 P3834: 可持久化线段树
 - 2: 洛谷 P3919: 可持久化数组
 - 3: 洛谷 P3402: 可持久化并查集
 - 4: 洛谷 P3168: 任务查询系统
 - 5: 洛谷 P3835: 可持久化平衡树
 - 6: 洛谷 P5055: 可持久化文艺平衡树
 - 7: 洛谷 P1383: 高级打字机
- 